

PyTrackingAnalysis — User Guide

Table of contents

1. [Overview](#)
2. [Environment setup with uv](#)
 - [Windows](#)
 - [macOS](#)
 - [Linux](#)
3. [Directory structure: experiments and Projects](#)
4. [The tracking_config.yaml reference](#)
5. [The desktop UI](#)
 - [Launching the apps](#)
 - [Analysis Hub](#)
 - [Config Editor](#)
 - [QC Viewer](#)
 - [Plot Editor](#)
6. [Running the pipeline from a notebook or script](#)
7. [Understanding the outputs](#)
8. [Projects: replicates and combined analysis](#)
9. [Removed regions: excluding flies you observed](#)
 - [What a removal means](#)
 - [Declaring a removal](#)
 - [What a removal does to the analysis](#)
 - [How removals are reported](#)
 - [Worked examples](#)
 - [Rules of thumb](#)
10. [Quick reference](#)

Scripts (saved analysis recipes) and the visual Script Editor have their own dedicated guide: [scripts_guide.md](#).

1. Overview

PyTrackingAnalysis is a Python pipeline for analysing insect-tracking data exported from DTrack. A single configuration file ([tracking_config.yaml](#)) describes the experiment — the tracking hardware, the experimental design, and how each physical tracking region maps to a treatment group. From that one file the pipeline can produce summary CSVs, statistical comparisons, publication-quality plots, and a multi-page PDF report. Several experiment directories can be grouped into a **Project** of replicates (§8) with a pooled Combined Analysis, project-level publication figures, and a Project Report; and any report can carry an optional **AI-written summary** (§5.2, §6).

There are several ways to drive the pipeline:

Interface	Best for
Analysis Hub (<code>pytrack-hub</code>)	Day-to-day use; loads experiments and Projects, runs analyses, shows plots in a tabbed dock
Config Editor (<code>pytrack-config</code>)	Authoring <code>tracking_config.yaml</code> + visual Script Editor for saved recipes
QC Viewer (<code>pytrack-qc</code>)	Per-tracker data-quality tables + XY / distance / timeline plots
Plot Editor (<code>pytrack-plots</code>)	Project-level publication figures (plotnine, vector SVG/PDF output)
Jupyter notebook (<code>Notebooks/SimpleTracker.ipynb</code>)	Exploratory work; custom plots
Python script / REPL	Automation; integration with other tools

2. Environment setup with uv

`uv` is a fast Python package manager that reads `pyproject.toml` and locks exact dependency versions. The project requires **Python 3.13 or later**.

2.1 Windows

Install uv

Open PowerShell and run:

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Restart the terminal so `uv` is on `PATH`.

Clone the repository and create the environment

```
git clone https://github.com/spletcher1/PyTrackingAnalysis.git
cd PyTrackingAnalysis

# Create a virtual environment using the Python version in .python-version
# (3.13)
uv sync
```

`uv sync` reads `pyproject.toml`, downloads and installs all dependencies into `.venv\`, and is idempotent — safe to run again after pulling updates.

Activate the environment (optional — `uv run` works without it)

```
.venv\Scripts\Activate.ps1
```

Run the UI

```
# With the environment activated:
pytrack                # Analysis Hub (default entry point)
pytrack-hub            # Analysis Hub (same as pytrack)
pytrack-config         # Config Editor
pytrack-qc             # QC Viewer

# Or without activating, using uv run:
uv run pytrack-hub

# With an explicit experiment (or Project) directory:
uv run pytrack-hub "C:\Users\you\Experiments\Trial1"
```

PyQt6 note on Windows: If a "platform plugin not found" error appears, install the Visual C++ Redistributable from Microsoft's website, then retry.

2.2 macOS

Install uv

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

Restart the terminal (or run `source ~/.zshrc` / `source ~/.bash_profile`).

Clone the repository and create the environment

```
git clone https://github.com/spletcher1/PyTrackingAnalysis.git
cd PyTrackingAnalysis
uv sync
```

Activate the environment (optional)

```
source .venv/bin/activate
```

Run the UI

```
# With environment activated:
pytrack                # Analysis Hub (default entry point)
```

```
pytrack-hub          # Analysis Hub (same as pytrack)
pytrack-config       # Config Editor
pytrack-qc           # QC Viewer

# Without activating:
uv run pytrack-hub

# With an experiment (or Project) directory:
uv run pytrack-hub /path/to/Trial1
```

macOS display server note: PyQt6 requires a display. Confirm XQuartz is not needed (native macOS Cocoa backend is used automatically).

2.3 Linux

Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
# Or via pip if curl is unavailable:
pip install uv
```

Restart the terminal or run `source ~/.bashrc`.

Install system Qt dependencies (if missing)

On Debian/Ubuntu:

```
sudo apt install libxcb-cursor0 libxcb-xinerama0 libxkbcommon-x11-0
```

On Fedora/RHEL:

```
sudo dnf install xcb-util-cursor libxkbcommon-x11
```

Clone the repository and create the environment

```
git clone https://github.com/spletcher1/PyTrackingAnalysis.git
cd PyTrackingAnalysis
uv sync
```

Activate the environment (optional)

```
source .venv/bin/activate
```

Run the UI

```
pytrack-hub                # Analysis Hub
uv run pytrack-hub         # works from any directory
uv run pytrack-hub /path/to/Trial1 # open a specific project
uv run pytrack-config      # Config Editor
uv run pytrack-qc /path/to/Trial1  # QC Viewer
```

Headless / SSH note: PyQt6 requires an X11 or Wayland display. If running over SSH, forward the display with `ssh -X` or use a VNC/RDP session. For fully headless processing, use the Python API (§6) instead — its legacy `batch_analyze` helper covers many folders at once.

Updating the environment

After pulling new commits:

```
git pull
uv sync      # installs / removes packages to match the updated lock
file
```

3. Directory structure: experiments and Projects

An **experiment directory** is the root folder for a single recording. It must contain `tracking_config.yaml` and a `data/` sub-folder with the DTrack export files. The pipeline creates `analysis/` and `qc/` automatically on first run. (Several experiment directories become replicates of a **Project** — see the Projects section below.)

```
MyExperiment/                ← experiment directory (pass this to
the UI)
├─ tracking_config.yaml      ← experiment configuration (required)
├─ removed_regions.yaml     ← regions you removed from the
analysis (optional, §9)
├─ data/                    ← DTrack export files (required)
│   └─ ExperimentName.xlsx  ← main DTrack workbook
│   └─ ExperimentName_Data_1.csv ← per-tracker CSV, one per tracking
region
│       └─ ExperimentName_Data_2.csv
│       └─ ...
│       └─ ExperimentName_BG.jpg ← background image (optional)
│       └─ LeftProgram.txt      ← stimulus program files (optional)
│       └─ RightProgram.txt
```

```

├── analysis/                                ← created by the pipeline
│   ├── ExperimentName_experiment_summary.txt
│   ├── ExperimentName_Summary.csv
│   ├── ExperimentName_Summary_Facet.csv
│   ├── ExperimentName_Stats.txt
│   └── ExperimentName_Notes.txt            ← run notes typed in at Run Analysis
(optional)
├── ExperimentName_plot_pi_facet.png
├── ExperimentName_plot_percentage_facet.png
├── ExperimentName_plot_transitions_facet.png
├── ExperimentName_plot_totaldistance_facet.png
└── qc/                                    ← created by the pipeline
    └── ExperimentName_data_quality.csv

```

Naming rules:

- The `.xlsx` file determines the experiment name. Every output file is prefixed with that name.
- The per-tracker CSVs must follow the pattern `<name>_Data_<N>.csv`. `N` is matched to tracking region `T_N-1` in the YAML (i.e. `_Data_1.csv` → region `T_0`).
- `tracking_config.yaml` must be at the **top level** of the experiment directory — not inside `data/`.

Projects (replicates of one design)

A **Project** is a directory with a `project.yaml` marker whose immediate subdirectories containing a `tracking_config.yaml` are its **Experiments** — replicates of one design (see [docs/adr/0005](#)):

```

MyProject/
├── project.yaml                            ← makes it a Project (name, notes, design,
scripts)
├── plot_specs.yaml                        ← project-level publication figure specs
├── analysis/                             ← Combined Analysis (pooled CSVs + stats +
AI narrative)
├── figures/                              ← project publication figures (SVG/PDF)
├── MyProject_report.pdf                   ← Project Report
├── Trial1/                                ← an Experiment (replicate)
│   ├── tracking_config.yaml
│   ├── data/ analysis/ qc/
│   └── Trial1_report.pdf
└── Trial2/ ...

```

`project.yaml`'s **design** section is the authority for everything the replicates share: the experiment type, design factors *and levels*, facet cutoffs and phase names, quality criteria (`min_transitions`, `min_movement`), and the counting-region **names** (aliases stay per-experiment). Opening the Project **hard-validates** every replicate's resolved config against it (a config omitting a key the type defaults still matches a design stating that default); region → treatment assignments, fly counts, and rigs may differ. The Create/Edit Project dialog edits the design and **Create experiment** scaffolds new replicate configs from it — so an empty Project is a valid starting point. Projects also have their own scripts: `project.yaml` can carry **Project Scripts** (`scripts:`) and centrally-held **Experiment Scripts** (`experiment_scripts:`); the Analysis card's script picker always includes the built-in **Report pipeline** and **Standard pipeline**. See [scripts_guide.md §8](#).

Flies from the same treatments are **pooled across replicates** for the combined plots, data, and statistics: the Combined Analysis stacks each replicate's *filtered* summaries with an **Experiment** column, and its statistics show the pooled per-fly Welch/Tukey tests beside a **linear mixed model** (treatment fixed, experiment random intercept) that accounts for between-replicate variation. A parent folder from the retired batch-over-experiments mode becomes a Project by writing a **project.yaml** into it (the Hub's **Create project** button does exactly that).

project.yaml may also hold two script sections (see §8.3): **scripts**: — **Project Scripts**, step lists of project-level actions — and **experiment_scripts**: — experiment-level scripts held centrally so one recipe serves every replicate without being copied into their configs.

4. The tracking_config.yaml reference

tracking_config.yaml has four possible top-level sections: **global**, **tracking_regions**, **counting_regions**, and **scripts**. **global** and **tracking_regions** are required for every experiment; **counting_regions** is required for the two-choice and counter tracking types; **scripts** is optional.

Creating a valid file

There are three equally valid ways to create one:

1. **Config Editor** (recommended) — **pytrack-config** gives you structured forms for every section, bulk region generation, and a live YAML preview, so the file is valid by construction.
2. **Copy an existing config** — copy a working **tracking_config.yaml** into the new experiment directory and edit it. Inside a Project, **Experiment configs...** scaffolds a design-conformant config for every experiment directory that lacks one; **Create experiment...** does it for a directory that does not exist yet, and **Initialize existing directory...** for one that exists without a config.
3. **Write it by hand** — any text editor works; the file is plain YAML.

Rules that make a file *valid*:

- The file must be named **tracking_config.yaml** (exact lowercase) and live in the experiment directory (next to **data/**, not inside it).
- An experimental design is **required** — an experiment will refuse to load without a parseable config.
- **tracking_type** must be one of the values in §4.1 (only a *missing* key falls back to **TRACKER**; a wrong value is an error).
- **TWOCHOICETRACKER** and **TWOCHOICECOUNTER** must define **exactly two** **counting_regions** entries.
- Every **counting_regions** entry must have an **alias** key.

Check at any time with **Validate YAMLs**, on its own row at the bottom of the Project panel's **Create/Load** card. With a Project selected it validates the **project.yaml** and every replicate's **tracking_config.yaml** in one pass; parse errors and semantic problems are reported per file in the Output and Errors tabs, with a total at the end.

4.1 **global** — required fields

```
global:
  tracking_type: TWOCHOICETRACKER # see table below
  tracking_rig: colosseum # see table below
```

experiment_type (recommended)

An **Experiment Type** is a named bundle that standardizes an assay end to end: it fixes the tracking type, the phases, and the required regions, constrains the rig, and produces a report tailored to that assay. Choose one and the fields it owns are supplied for you — you do **not** write them in the file.

```
global:
  experiment_type: Valence # fixes tracking_type + phases; constrains
the rig
  tracking_rig: colosseum # you choose: arena_max or colosseum
```

Value	What it fixes
Valence	Two-choice light-preference assay. Tracking type TWOCHOICETRACKER ; Light/NoLight counting regions (in that order, so positive PI = light-preference); phases Acclimation (0–10) / Experiment (10–70) / Cooldown (70+) ; rig must be arena_max (36 regions, T_0–T_35) or colosseum (24 regions, T_0–T_23); calibration from the rig preset only.

For a type whose plate is fixed by the rig (like Valence), the Config Editor lays out the exact tracking regions when you choose the rig — 36 rows for Arena Max, 24 for Colosseum — and a typed config is checked to have exactly that set. You still assign each region's treatment; the region names and count are fixed.

Rules for a typed experiment:

- The file omits **tracking_type** — it comes from the type; a conflicting value is an error. **facet_cutoffs** is an **editable default** (10, 70 for Valence): it is written to the file and you may change it. The same goes for **facet_labels**, the phase names (Acclimation, Experiment, Cooldown for Valence): written to the file by default, yours to rename.
- Low-transition exclusion** (Valence only, **min_transitions**, default 5, editable in the Config Editor): a fly with fewer than this many transitions during the **primary phase** (the Experiment phase — the second facet window, or the only one) is excluded from every result — figures, summary measures, statistics, and the summary CSVs. A fly with *no* data in that window counts as excluded too. Set **min_transitions: 0** to turn the exclusion off. Excluded flies are listed in the report and in ***_Excluded.csv** (written even when empty), and still appear in data-quality output. See [docs/adr/0003](#).
- Low-movement flag** (Valence only, **min_movement**, default 140 mm/min, editable in the Config Editor): a fly averaging less than this movement during the **first** facet window (Acclimation at default cutoffs) is flagged as potentially an issue — reported, **never removed**. Flagged flies stay in every figure, statistic, and CSV, marked by a **LowMovementFlag** column in the saved summary CSVs and listed in the report. When more than half of the analysed flies are flagged, the whole experiment is

noted as potentially an issue on the report cover and in `*_Stats.txt.min_movement: 0` turns the flagging off.

- **Removed regions** (any experiment type, Valence or Custom): tracking regions *you* declare out of the analysis — a fly that died partway through, escaped, or a well that was empty. Nothing in the data reveals these, so you enter them by hand: in the Hub (**Project** → **Removed regions...**), by editing `removed_regions.yaml` at the experiment directory's root, or in bulk from a `removed_regions.csv` spreadsheet that a Batch Run applies before it starts. Every fly in a removed region is excluded exactly as an automatically excluded one is, and is listed with your reason in both reports and in `*_Excluded.csv`. §9 covers this in full; see also [docs/adr/0010](#).
- A typed config is validated **at load** and **fails hard** on any violation (wrong rig, missing Light/NoLight, a disallowed override), rather than crashing mid-analysis.
- Omitting `experiment_type` entirely is a **Custom** experiment: the freeform, `tracking_type`-driven behavior described below, unchanged. Existing configs keep working as-is.

The fastest way to start is to make the Project first: **Project** → **Create/Load** → **Create project...** in the Analysis Hub, where you choose the Experiment Type, the design factors and levels, the facets (default 10, 70) and the quality criteria **once**, for every replicate. Then **Project** → **Experiments** → **Create experiment...** scaffolds each replicate from that design — its `tracking_config.yaml` plus the `data//analysis//qc/` folders — and asks only for the directory name, because everything else is the Project's. The Hub then offers two ways to finish the scaffold: **Edit config...** opens it in the Config Editor, **Copy config from...** replaces it with the config of a replicate that already works (checked against the design before it is written). For Valence the Config Editor lays out the plate as soon as you pick the rig — 36 regions for Arena Max (with the first 18 X-flipped) or 24 for Colosseum, plus the Light/NoLight aliases. You then assign each region's treatment and drop the DTrack export into `data/`. You can also pick the Experiment Type from the dropdown at the top of the Config Editor's **Global** tab. See [docs/adr/0001](#) and [0002](#).

The rest of §4.1 describes the fields a **Custom** experiment sets directly.

tracking_type

Selects the analysis mode. Choose the one that matches how DTrack recorded your data. The value is upper-cased before matching, so `twochoicetracker` works too. If the key is missing entirely, `TRACKER` is assumed; an unrecognized value is an error that lists the valid choices.

Value	Description
<code>TRACKER</code>	Position tracking only — computes distance and speed
<code>TWOCHOICETRACKER</code>	Two-region choice assay — computes PI, percentage time, transitions
<code>XCHOICETRACKER</code>	Multi-region assay along a linear axis — computes adjusted X position
<code>PAIRWISEINTERACTIONTRACKER</code>	Proximity-based interaction scoring between pairs of tracked animals
<code>COUNTER</code>	Frame-count based occupancy (no continuous position)
<code>TWOCHOICECOUNTER</code>	Counter-based two-region choice — PI and percentage

Value	Description
<code>PAIRWISEINTERACTIONCOUNTER</code>	Counter-based pairwise interaction scoring

tracking_rig

Selects the hardware calibration preset. `fps` and `mm_per_pixel` are set automatically from the preset; all other parameters use the defaults shown below. The value is matched case-insensitively with spaces/hyphens treated as underscores (`Arena Max` → `arena_max`), and the common misspelling `colloseum` is accepted for `colosseum`. An unknown rig name is not an error — the tracking type is still applied but all calibration values stay at their generic defaults, so double-check the spelling.

Value	mm per pixel	Notes
<code>small_arena</code>	0.056	
<code>arena_max</code>	0.145	
<code>colosseum</code>	0.108	
<code>obscura</code>	0.131	
<code>movie</code>	—	You must supply <code>fps</code> and <code>mm_per_pixel</code> manually

`fps` for all hardware rigs is read from the timestamps in the DTrack export rather than set to a fixed value. Only the `movie` rig requires an explicit `fps`.

4.2 global — experimental design

```
global:
  experimental_design_factors:
    feeding: [Starved, Control]
    sex:     [Male, Female]
```

List every factor and its levels. These are used for axis labels and plot titles. The actual assignment of factors to each physical tracking region is made in `tracking_regions` (see §4.4).

4.3 global — optional fields

```
global:
  # Split the recording into time phases for faceted plots and statistics.
  # [10, 70] creates three phases: 0–10 min, 10–70 min, 70+ min.
  # Remove this key entirely to disable faceted analysis.
  facet_cutoffs: [10, 70]

  # Optional names for those phases, one per phase (= one more than the
  # number
  # of cutoffs). Used in faceted figures, the report, and project
```

```

summaries;
# omit it to fall back to the Experiment Type's defaults (for Valence:
# Acclimation / Experiment / Cooldown) or plain minute ranges.
facet_labels: [Acclimation, Experiment, Cooldown]

# Only needed for the 'movie' rig, or to override a hardware preset.
fps: 30
mm_per_pixel: 0.108

# Smoothing window for speed calculation (seconds). Default: 1
speed_window_seconds: 1

# Speed range [min, max] mm/s that defines micro-movement. Default: [0.2,
2]
micromove_speed_mm_sec: [0.2, 2]

# Minimum speed (mm/s) to count as walking. Default: 2
walking_speed_mm_sec: 2

# Minimum continuous resting duration (minutes) to count as sleep.
Default: 5
sleep_threshold_min: 5

# Distance thresholds (mm) for interaction detection.
# Only used with PAIRWISEINTERACTIONTRACKER / PAIRWISEINTERACTIONCOUNTER.
interaction_distances: [8]

```

How overrides are interpreted: the rig preset is applied first, then any of the recognized parameter keys present in `global`: override the preset value. Exactly these keys are recognized as overrides — `fps`, `mm_per_pixel`, `speed_window_seconds`, `micromove_speed_mm_sec`, `walking_speed_mm_sec`, `sleep_threshold_min`, `interaction_distances`. Any other key in `global`: is carried along but ignored by the parameter system, so a typo like `walking_speed` will not error — it simply won't take effect.

`facet_cutoffs` is read separately (not a parameter override): it becomes the default for every faceted plot, summary, and statistics run, and is inherited by script steps whose own `cutoffs` field is blank (see [scripts_guide.md](#)). `facet_labels` names those phases; it must list exactly one name per phase (`cutoffs + 1`) and is validated against `facet_cutoffs`.

How time windows are defined. Every window — a `facet_cutoffs` phase or an explicit `range_minutes=(start, end)` — is *half-open*: it contains rows where `start ≤ Minutes < end`. `facet_cutoffs: [10, 70]` therefore produces the phases `[0, 10)`, `[10, 70)`, and `[70, ∞)`, and each frame belongs to exactly one of them. `range_minutes=(0, 0)` still means "the whole recording".

Accumulated quantities such as `TotalDistance` are computed *within* the window. Each per-frame step is credited to the window containing the frame it arrives at, so a step spanning a cutoff is counted once, in the later phase. The phase totals of a faceted summary therefore add up to the flat summary total exactly.

Which frames count. `DataQuality` is reported, not enforced: `TotalDistance` sums every step in the window, including steps into and out of frames where tracking was lost. Those frames carry real coordinate jumps rather than blanks, so on real recordings they inflate `TotalDistance` by up to ~19 % for the worst animal, in proportion to how poor the tracking was. The summary therefore also reports

TotalDistanceHighQualityOnly, which discards any step with a lost endpoint, alongside **PercHighQuality**. Compare the two on your own data before reporting distances; if tracking quality correlates with treatment, part of a distance difference between arms is a tracking artifact rather than behaviour.

A frame whose speed is undefined — a duplicate or stalled timestamp, or the lead-in rows of the rolling speed window — cannot be classified as walking, micro-moving or resting. Such frames are counted in **PercUnmeasurable** and excluded from the denominator of the other four fractions, so those four still sum to 1 over the frames that were actually measurable. Previously they fell through to **PercResting**, making "the animal was still" indistinguishable from "we could not tell". A window in which *every* frame is unmeasurable reports **NA** for the activity fractions rather than 100 % resting.

Windows used to include *both* endpoints, so a frame landing exactly on a cutoff was counted in the phase before it and the phase after it. How much that mattered depends on the time base. With **fps: 0** (the default for every rig preset) minutes come from the DTrack **MSec** column and essentially never land on an exact cutoff, so those results were already correct. With an explicit **fps**, minutes are $\text{Frame} / (\text{fps} \times 60)$ and hit integer cutoffs exactly — one frame per boundary was double-counted, inflating a faceted total by roughly one sampling interval of movement per cutoff. **XChoiceTracker**'s **TotalXDistance_mm** had the mirror-image problem and lost one step per window.

4.4 tracking_regions

One entry per DTrack tracking region. Region names must match the naming scheme used in the CSV files: region **T_0** corresponds to **_Data_1.csv**, **T_1** to **_Data_2.csv**, and so on.

```
tracking_regions:
  T_0:
    experimental_factors: Starved, Female # comma-separated, order must
    match                                                         # experimental_design_factors
    order
    x_location_multiplier: 1 # set to -1 to flip the X axis
    y_location_multiplier: 1 # set to -1 to flip the Y axis
  T_1:
    experimental_factors: Control, Male
    x_location_multiplier: 1
    y_location_multiplier: 1
    # ... one entry per region
```

How each field is interpreted:

- experimental_factors** — a free-form string that becomes the region's *Treatment* label used in grouping, plots, and statistics. For multi-factor designs list the levels comma-separated in the same order as **experimental_design_factors** (e.g. **Starved, Female**). Regions sharing the same string are analysed as one group. A missing key means the region has an empty treatment (it still loads, but groups as blank — the experiment summary lists such regions as *(unassigned)*).

- `x_location_multiplier` / `y_location_multiplier` — correct for physical differences in camera orientation between regions. Use `1` for no correction and `-1` to mirror an axis. Only `1` and `-1` are meaningful: any other value (or a missing key) is silently treated as `1`.

The experiment summary text file (§7) includes a formatted description of the loaded design — factors, region assignments, non-unit multipliers, counting regions, and cutoffs — so you can verify the YAML was interpreted the way you intended.

4.5 counting_regions

Maps the region labels used inside the DTrack data to canonical choice names. Required for the counter types (`COUNTER`, `TWOCHOICECOUNTER`, `PAIRWISEINTERACTIONCOUNTER`) **and** for `TWOCHOICETRACKER` — both two-choice types are validated to have **exactly two** counting regions and will refuse to load otherwise.

```
counting_regions:
  Light:
    alias: Light, LL, L      # any of these strings in the data = "Light"
  NoLight:
    alias: NoLight, NL, N
```

How it is interpreted: each key (`Light`, `NoLight`) is a canonical characteristic name; `alias` is a comma-separated list of the raw region labels that map to it (whitespace around each alias is stripped). Matching is exact and case-sensitive after stripping. Every entry **must** have an `alias` key — omitting it is a config error that prevents the design from loading.

4.6 scripts — saved analysis recipes (optional)

Saved, re-runnable step lists that the Hub's **Scripts** card executes. Normally you author these visually in the Script Editor rather than by hand:

```
scripts:
- name: nightly
  steps:
- action: load_experiment
  params: {path: '.', force_preprocessing: false}
- action: run_analysis
  params: {facet: true, cutoffs: ''}
```

Each script is `{name, steps}`; each step is `{action, params}` where `action` is one of the registered action keys and `params` matches that action's schema. Inside a **Project**, experiment-level scripts can instead be held centrally in `project.yaml`'s `experiment_scripts`: section, and the Project Script action `run_in_experiments` runs a named experiment script in every replicate — resolved from the Project's central section first, falling back to a script of that name in each replicate's own `tracking_config.yaml` (which is how a legacy `batch` script still runs; see §8.3). See [scripts_guide.md](#) for the full action reference, validation rules, and hand-editing guidance.

4.7 Complete minimal example

```
global:
  tracking_type: TWOCHOICETRACKER
  tracking_rig: colosseum
  experimental_design_factors:
    genotype: [WT, Mutant]
    facet_cutoffs: [10, 60]

tracking_regions:
  T_0:
    experimental_factors: WT
    x_location_multiplier: 1
    y_location_multiplier: 1
  T_1:
    experimental_factors: Mutant
    x_location_multiplier: 1
    y_location_multiplier: 1
```

5. The desktop UI

PyTrackingAnalysis ships four independent PyQt6 apps, each launched as its own window. They share a common pyflic-style theme (category-colored cards, top bar, PlotDock) so the visual language is consistent across all of them.

Command	Window	Purpose
pytrack-hub (or just pytrack)	Analysis Hub	Day-to-day Project driver: manage replicates, load experiments, run analyses, build combined results, render figures in a tabbed dock, launch Config + QC + Plot Editor
pytrack-config	Config Editor	Structured editor for tracking_config.yaml + visual Script Editor for saved recipes
pytrack-qc	QC Viewer	Per-tracker data-quality table + XY / distance / quality-timeline plots
pytrack-plots	Plot Editor	Publication figures (project-level): live-edit pooled plots' style and content, save vector output (SVG/PDF) for Illustrator

5.1 Launching the apps

```
# With the environment active:
pytrack                    # Hub (shorthand)
pytrack-hub               # Hub
pytrack-hub /path/to/MyProject # Hub, pre-loaded Project
```

```

pytrack-config                                # Config Editor (opens last-used
or ./tracking_config.yaml)
pytrack-config /path/to/MyExperiment          # Config Editor, pre-loaded
tracking_config.yaml

pytrack-qc /path/to/MyExperiment              # QC Viewer, pre-loaded experiment

pytrack-plots /path/to/MyProject              # Plot Editor (Project directories
only)

# Without activating, through uv:
uv run pytrack-hub
uv run pytrack-config /path/to/Trial1

# Dev shortcut (equivalent to the console scripts above):
python -m pytrackinganalysis hub /path/to/Trial1
python -m pytrackinganalysis config /path/to/Trial1
python -m pytrackinganalysis qc /path/to/Trial1
python -m pytrackinganalysis plots /path/to/MyProject

```

Desktop launcher / taskbar icon (Linux). The apps set their own window icon (a fly in a tracking reticle), but on Wayland/GNOME the *taskbar* icon comes from a `.desktop` entry. Install entries for the desktop apps once per environment with:

```
uv run pytrack-install-desktop
```

This also adds the apps to your desktop's application launcher. Re-run it if you move the project or recreate `.venv` (the entries embed absolute paths).

All four apps persist the light/dark theme choice to `~/.config/pytrackinganalysis/ui.json`. Recent projects — and the last-used AI provider/model — are tracked there too.

Every app's top bar and most cards carry a **?** button that opens the in-app help on that topic — the same ground as this guide, one screen at a time.

5.2 Analysis Hub (`pytrack-hub`)

The Hub's layout (see [docs/adr/0007](#) and [docs/adr/0009](#)) is a **tile strip** across the top — six compact live-status tiles: **Batch** • **Project** • **Analyze** • **Plots** • **Scripts** • **AI** — with the **output area at full width** underneath. The first two tiles work on containers (a Batch or a Project); the tiles after them act on the loaded experiment. A tile shows only status (the project's name and replicate health, the loaded experiment's fly counts, whether analysis is faceted, ...); **clicking it drops an anchored panel** holding all of that area's controls. One panel is open at a time; **Esc** or clicking anywhere else closes it, and **running tasks leave panels in place** while the streaming log and plots remain visible below. Tiles never move or hide: a tile whose subject is missing is dimmed with a hint — greyed surface, muted title, greyed icon — and so are the cards inside its panel, so the state reads the same from the strip and from the open panel. The panel still contains exactly the control that fixes the missing state (the dimmed Analyze tile opens the panel that tells

you to load an experiment first), and a dimmed tile or card stays clickable throughout. **Batch** and **Project** are never dimmed — the two entry points are always available, and say their state in words ("no project - open or create one") rather than by greying. The four experiment-level tiles - Analyze, Plots, Scripts, AI - dim together until a replicate is loaded.

The Hub is **Project-first** ([docs/adr/0008](#)): an experiment is loaded only by double-clicking its row in the Project panel's replicates table, so there is one subject at a time because there is one way to change it. There is no Experiment tile — the Project tile reports the loaded experiment on its second line, and the **status readout** filling the strip right of the AI tile spells the same state out in full: project name and type, path, replicate and analysis counts, and the loaded experiment (design factors in its tooltip).

The panels:

- **Batch** — lights when the selected folder has Projects anywhere beneath it (the selection names a Batch or a Project — still exactly one working container). Discovery is **recursive and stops at each Project**, so [Sept2026/ProjA](#) and [Archive/2025/ProjC](#) are both found and a row's name is its path inside the batch folder. A checkable **projects table** lists every Project with its usable replicate count ([3/5](#)), its report status, and any **blocked experiments** in red; all are checked by default except a Project with nothing the run can use. **Double-click a row to select that Project** — an ordinary selection change, so there is no "up to batch" button; **right-click** it to fix a blocked experiment or edit its removed regions, and **Rescan folder** walks the tree again after changes made outside the app. A **Script** picker names the **designated Project Script** (default: the built-in **Report pipeline**), an opt-in **AI narrative of the batch** checkbox (§8.5), and **Run batch** runs the script in every checked Project (§8.5) — after a **review window** that states the target list, offers to file any recording still sitting loose at an experiment root, and previews (or declines) the removal sheet.
- **Project** — three cards, in the order the work happens: **Create/Load** (the Project itself), **Experiments** (its replicates), **Analysis** (what you do with them). None repeats the word Project, because the panel is already titled with it.
 - **Create/Load** — the read-only path box shows the selected folder (hover it for the full path), and there is one button per state a folder can be in: **Open Project** for a directory that already has a [project.yaml](#) (picking the one already open re-reads it from disk — there is no separate Reload button, the picker is the reload); **Create project...** for a Project that does not exist yet (you choose where it goes, name it, and fill in the design); **Initialize existing directory...** for a directory that exists but has no [project.yaml](#) (it keeps its name, its subdirectories become the replicates, and the design is inferred from the first one that has a config). **Edit config...** reopens the Project editor on the loaded [project.yaml](#), and reads **Create config...** when the selected folder is not a Project yet; pointing the Hub at a folder that is *itself* an experiment makes it offer the **parent** instead, so the experiment becomes a replicate — a [project.yaml](#) written beside a [tracking_config.yaml](#) would be a Project with nothing to load. Full width on its own row below them, **Validate YAMLs** checks the open Project's [project.yaml](#) and every replicate's [tracking_config.yaml](#) in one pass. A summary line under the buttons describes what is loaded. There is no tracking-config picker here — [project.yaml](#) is fixed at the Project root, and each experiment's [tracking_config.yaml](#) lives one level down (use **Experiment configs...** in the Experiments card); QC is experiment-level and opens on load.
 - **Experiments** — the replicates themselves: a table with per-replicate status (**Experiment, Config, Flies, Excluded, Flagged, Report**; **double-click a row to load that replicate** — this is the only way to load an experiment, and it runs QC as it loads), and the same three cases one

level down — **Create experiment...** (the replicate does not exist; it is scaffolded from the project design and only its name is asked for), **Initialize existing directory...** (the directory is in the Project but has no config; any loose recording is filed into **data/**, every other loose file into **extra_files/**, and the config is scaffolded from the design), and **Experiment configs...** (create or edit each replicate's config in bulk). All three wait for a Project, because a replicate's design is inherited from **project.yaml**. Subdirectories that hold no **tracking_config.yaml** are listed too, in italics with **Config: missing** — they are not replicates until they have one.

- **Analysis** — the project-level actions over every replicate: **Create report** or **Update report** (the label follows whether **<project>_report.pdf** exists), **Plot editor...**, **AI narrative...**, **View reports** (opens the project report and every replicate report in the desktop's PDF viewer; enabled only once both kinds exist), and **Removed regions...** (§9.2.1). At the bottom, a **Script** picker with **Run script / Edit scripts...** (§8.3; the built-in **Report pipeline** and **Standard pipeline** are always available).
- **Analyze** — a **Faceted** checkbox (its label shows the configured cutoffs, and the buttons it applies to gain a (**facet**) suffix while it is on) over **Run Analysis**, **Run QC only**, **Create PDF Report**, **Summarize** and **Run pairwise comparisons** for the loaded experiment. Run Analysis and Create PDF Report ask for optional run notes. All tasks run on a background thread; stdout/stderr streams to the **Output** tab in real time.
- **Plots** — dynamically populated with the faceted plots valid for the loaded tracking type (**plot_pi_facet**, **plot_totaldistance_facet**, etc.). Each click adds a new tab to the PlotDock. Toggle **Interactive plots** in the top bar to switch between static PNG rendering (fast) and a live canvas with pan / zoom / save toolbar.
- **Scripts** — lists saved analysis recipes from the active YAML's **scripts:** section. **Run Script / Run All** executes them and routes each step's log output to the Output tab and each figure to a PlotDock tab. Author scripts from the Config Editor (see 5.3 and [the scripts guide](#)).
- **AI** — **AI summary...** opens a dialog to pick a provider (Anthropic or OpenAI) and model, then writes a one-page, clearly-labeled **AI Summary** of the loaded experiment's analysis to **analysis/<name>_AI_Summary.txt** and rebuilds the report PDF to embed it. The action is offered only when an API key is present (**ANTHROPIC_API_KEY** / **OPENAI_API_KEY** in a **.env** file or the environment); the model dropdown refreshes itself monthly from the providers, with a manual refresh button. Re-running the analysis deletes the saved summary so stale prose never sits beside fresh figures — regenerate it afterwards if wanted. A failed call shows an error and never blocks the report.

There is **no Tools tile: Validate YAMLs** moved to the bottom of the **Create/Load** card, opening a folder is the file manager's job, and matplotlib's cache is now cleared automatically every time the Hub closes.

The output area (below the strip)

- The first tab is always **Output** — the chronological log of everything the Hub does.
- The second tab is **Errors** — a permanent tab that collects only warnings and errors (failed tasks with tracebacks, skipped or failed replicates, YAML validation problems, dismissed warning pop-ups, ...) so they can't get lost in the normal output. When issues arrive while you're on another tab, the tab title shows an unseen count, e.g. **Errors (3)**; viewing the tab resets it.
- Every plot or artifact opens as an additional closable tab. Three buttons in the tab-bar corner clear things separately: **Clear Analysis Tabs** closes every plot and artifact tab at once and returns to the

Output tab (Output and Errors are never closed), while **Clear Output** and **Clear Errors** empty those two logs.

5.3 Config Editor (**pytrack-config**)

Structured editor for **tracking_config.yaml** with three tabs wrapped in a Card:

- **Global** — drop-downs for **tracking_type** and **tracking_rig**; a table for experimental-design factors; optional facet cutoffs; text fields for each parameter override.
- **Tracking regions** — one row per region. **Generate N regions** bulk-fills **T₀...T_(N-1)**. X/Y multipliers are restricted to **1 / -1**.
- **Counting regions** — treatment label → DTrack aliases.

A **YAML preview** Card below the tabs renders the live serialization for trust / debugging; an amber ● dirty indicator surfaces unsaved edits.

Visual Script Editor

Open via the scripts icon in the top bar. A non-modal window with three panes:

- **Palette** (left) — category-grouped tile list of the registered actions (**load_experiment**, **filter_by_quality**, **filter_by_region**, **run_qc**, **run_analysis**, **summarize**, **run_pairwise_comparisons**, **create_report**, plus one plot action per plot valid for the project's tracking type). Double-click to add a step.
- **Canvas** (center) — ordered step cards with the action's icon, a parameter-summary chip, and move-up / move-down / delete buttons.
- **Inspector** (right) — dynamic form for the selected step with widgets derived from the action's param schema (spinbox, combo, line edit, checkbox, browse-for-path, comma-list). Red banner reports validation errors inline.
- **Preview** (bottom) — live YAML of the current script.

Scripts are stored under the **scripts:** key of the surrounding **tracking_config.yaml**. The Hub's **Scripts** card reads the same file, so saving in the Script Editor makes scripts immediately runnable from the Hub.

Opened on a Project's **project.yaml** (Project panel → Analysis card → **Edit scripts...**), the same editor gains a **level switcher**: **Project scripts** (the project-action palette — §8.3) and **Experiment scripts** (the familiar experiment palette, held centrally for every replicate).

Full documentation — every action and its parameters, faceting rules, and the two-level Project scripting model (§8.3) — lives in [scripts_guide.md](#).

5.4 QC Viewer (**pytrack-qc**)

- Left pane — **Trackers** table with columns **Tracker**, **HighQuality**, **NotFound**, **Indiscernible**, **StartMinutes**, **EndMinutes**. Rows tint green, yellow, or red against the experiment's QC cutoff, with the exact thresholds shown under the table; a filter box narrows by tracker name.
- Right pane — **PlotDock** that updates when you select a tracker with four tabs:

- **XY trajectory** — RelX/RelY scatter coloured by time (viridis).
- **Total distance over time** — cumulative **Dist_mm** vs **Minutes**.
- **X / Y vs time** — stacked RelX(t), RelY(t) line plots.
- **Data quality timeline** — per-frame **DataQuality** category plotted as a time series so bad-tracking regions jump out visually.
- **Export data_quality.csv** writes the full table to disk for external review.

5.5 Plot Editor (**pytrack-plots**)

Publication figures for Valence experiments (see **docs/adr/0004**), rendered by **plotnine** — a separate path from the PDF-report figures that shares the same summarized, exclusion-filtered data.

- **Project-level tool.** Open a **Project directory** (or launch from the Hub's Analysis card): the four faceted plots show all flies **pooled across replicates** (built from the replicates' filtered summaries), and a **Mark experiments** option gives each replicate its own point shape with a legend. **plot_specs.yaml** and **figures/** live at the **project root**. Opening a replicate directory redirects up to its Project; a standalone experiment is refused with guidance to create a Project around it first.
- **Two-layer model.** A named **Plot Style** holds the look shared across plots (figure size in mm, theme, font, geometry — jittered dots, boxplots, or both — point/mean styling, line weight for axes/ticks/borders, facet strip style: plain text or the ggplot-default bordered grey box with a choosable fill, an optional panel background color, and treatment colors); a per-plot **Plot Spec** holds content (labels, facet and treatment inclusion/order/renames, y-limits, reference line, independent per-facet y axes (**free_y**, the default for movement and transitions), and optional per-facet **p-value brackets** — Welch's t-test for two treatments, Tukey HSD beyond, the same policy as Stats.txt). Both persist in **<project>/plot_specs.yaml**, written only by this app.
- **Save style as...** captures the current look under a name so subsequent plots come out identical; **Set as project default** makes it the style the app auto-loads for this project.
- **Save SVG... / Save PDF...** write vector files (default **<project>/figures/<plot>.svg**). SVG text stays *editable text* in Adobe Illustrator (**svg.fonttype='none'**); PDF embeds TrueType fonts. The live preview is rendered from the same figure object that saving uses, so the file always matches the screen.
- **Headless re-render:** **Project(project_dir).render_figures(formats=("svg", "pdf"))** regenerates every figure defined in **plot_specs.yaml** without the app — also available as the **render_publication_figures** Project Script action.

6. Running the pipeline from a notebook or script

The analysis pipeline is also available as a Python API. This is the approach used in **Notebooks/SimpleTracker.ipynb**.

Setup

```
import warnings
warnings.filterwarnings("ignore")

# PyTrackingAnalysis is installed as a package by `uv sync`, so import it
by its
```

```
# full dotted path. This works from any working directory – no os.chdir
needed.
from pytrackinganalysis.Experiment import Experiment, batch_analyze
```

Single experiment

```
# Pass the experiment directory – the one containing tracking_config.yaml
and data/
exp = Experiment("./Data/Trial1/")

# To analyse a project against a second config in the same directory, name
it.
# Without this the filename tracking_config.yaml was joined on
unconditionally,
# so an alternative config could be validated but never actually used.
exp_alt = Experiment("./Data/Trial1/", config_path="alt_config.yaml")

# Human-readable summary of the loaded experiment
print(exp)

# Detailed per-tracker overview (also saved to analysis/)
exp.experiment_summary()

# Data quality report (also saved to qc/)
exp.qc(cutoff=0.9)      # cutoff = minimum fraction of valid frames per
tracker

# Save summary CSVs (flat + faceted) to analysis/
exp.save_summary()

# Run pairwise statistical comparisons and save results to analysis/
exp.stats()

# Save plots to analysis/ as PNG files
exp.save_plots()

# Build a multi-page PDF report in analysis/
exp.create_report()

# Optionally attach run notes – rendered near the top of the report and
saved
# as <Experiment>_Notes.txt in analysis/ (the Hub prompts for these when
you
# press Run Analysis / Create PDF Report; blank clears saved notes).
exp.create_report(notes="Pilot run; lights at 50% intensity.")

# Optional AI Summary (needs ANTHROPIC_API_KEY or OPENAI_API_KEY in .env or
# the environment). Writes analysis/<name>_AI_Summary.txt; the report
embeds
# it while that file exists, and run_analysis() deletes it (it describes a
# single analysis run).
```

```
exp.generate_ai_summary("anthropic")      # or "openai"; model=... to
pick one
exp.create_report()                      # now carries the AI
Summary section

# — OR run the complete pipeline in one call:

exp.run_analysis()                      # summary → qc → save_summary → save_plots → stats
exp.create_report()                    # PDF report (separate call so you can skip it)
```

For a **Valence Experiment** the report opens with type-specific sections before the generic figures: the **headline result** (per-animal PI during the Experiment phase — the phase the primary result is read from — with a pairwise-comparison table), **preference over time** (sliding-window PI, treatment mean $\pm$ SEM, phase boundaries marked), and **emergence & persistence** (each animal's PI across phases plus the within-animal change from Acclimation to Experiment). A Custom Experiment gets the generic report unchanged.

Accessing individual plots interactively

Inside a Jupyter notebook with `%matplotlib inline`:

```
# These display the plot inline in the notebook (not saved to disk).
exp.arena.plot_pi()
exp.arena.plot_pi_facet(cutoffs=[10, 70])
exp.arena.plot_percentage_facet(cutoffs=[10, 70])
exp.arena.plot_transitions_facet(cutoffs=[10, 70])
exp.arena.plot_totaldistance_facet(cutoffs=[10, 70])
```

Every `*_facet` method requires `cutoffs`, either explicitly or via `facet_cutoffs` in the config; calling one without them raises rather than guessing. They previously defaulted to a literal `(10, 70)`, so a project with no cutoffs configured produced plots split at 10 and 70 minutes sitting beside whole-recording p-values, with nothing marking the discrepancy.

The available plot methods depend on `tracking_type`:

tracking_type	Plot methods
TRACKER	plot_totaldistance_facet
TWOCHOICETRACKER	plot_pi_facet, plot_percentage_facet, plot_transitions_facet, plot_totaldistance_facet
TWOCHOICECOUNTER	plot_pi_facet, plot_percentage_facet
XCHOICETRACKER	plot_adjusted_x_position_facet, plot_totaldistance_facet
PAIRWISEINTERACTIONTRACKER	plot_interactions_facet, plot_totaldistance_facet
PAIRWISEINTERACTIONCOUNTER	plot_interactions_facet

Projects from Python

The `Project` object (§8) mirrors everything the Hub's Project panel does:

```
from pytrackinganalysis.project import Project, create_project_file

# One-time: turn a directory of replicate experiment directories into a
# Project by writing the project.yaml marker (idempotent; preserves keys).
create_project_file("./MyStudy/", name="MyStudy")

prj = Project("./MyStudy/")          # discovers replicates, validates the
design                                # the replicate directories
print(prj.experiment_names)          # non-fatal differences (rigs, cutoffs,
print(prj.warnings)                  ...)

prj.run_all()                        # run_analysis + report in every
replicate
prj.build_combined_analysis()         # pooled CSVs + pooled/mixed stats →
<project>/analysis/
prj.render_figures(formats=("svg",))  # publication figures from
plot_specs.yaml
prj.generate_ai_summary("anthropic")  # optional AI narrative (embedded
below)
prj.create_report()                  # <project>/<name>_report.pdf
```

`Project(...)` raises with a per-replicate problem list when a replicate does not match the design in `project.yaml` (or, without a `design:` section, when replicates disagree on experiment type or design factors/levels). `build_combined_analysis()` deletes any saved AI narrative — like `run_analysis()`, the narrative describes a single build.

Legacy batch processing from a script

```
results = batch_analyze("./Data/")

for path, status in results.items():
    tag = "OK  " if status == "ok" else "FAIL"
    print(f" {tag} {path}")
    if status != "ok":
        print(f"      {status}")
```

`batch_analyze` — the legacy Python batch helper, experiment-level and pre-Project — scans every immediate sub-directory of the supplied path, identifies valid experiment directories, runs `exp.run_analysis()` and `exp.create_report()` on each, and returns a `{path: "ok" | error_message}` dictionary. It is unrelated to the Hub's **Batch Runs** (§8.5), which run a designated Project Script across many Projects.

7. Understanding the outputs

All outputs are written relative to the experiment directory (project-level outputs to the Project root — see the last table below).

analysis/ — main results

File	Contents
*_experiment_summary.txt	Rig settings, parameters, a formatted description of the experimental design (factors, region assignments, non-unit multipliers, counting regions, cutoffs), data quality overview, per-tracker table
*_Summary.csv	Per-tracker summary statistics (one row per tracker). For Valence, a LowMovementFlag column marks flies flagged by the low-movement check (they remain in the data)
*_Summary_Facet.csv	Same, split into the time phases defined by facet_cutoffs
*_Excluded.csv	Every fly left out of the analysis — name, region, treatment, transition count in the primary phase (Valence), and a Reason : your own removal (Removed: dead at ~20 min), the low-transition criterion, or both on one row. Written even when no fly was excluded, so absence never needs interpreting
*_Stats.txt	Pairwise statistical comparisons across treatment groups: independent two-sample Welch's t-test (unequal variance) when there are exactly two treatment levels, Tukey HSD when there are three or more. Each line carries both groups' N, mean and SD, and any trackers dropped for having no numeric value in the window are counted explicitly. Faceted runs append a note stating how many uncorrected tests were run and the Bonferroni-adjusted threshold. Pass equal_var=True to run_pairwise_comparisons for the classic Student's test.
plot.png	One PNG per plot type, named after the plot method
*_AI_Summary.txt	(Optional) The saved AI Summary; provenance (provider, model, date) on the first line. The report embeds it while this file exists; every run_analysis() deletes it so it can never describe a stale run
ai_narrative.md	(Optional) The same AI Summary as Markdown, under a fixed, name-independent filename so **/ai_narrative.md finds every narrative in a tree. Written and deleted with its .txt sibling
<name>_report.pdf	Written to the experiment directory root (beside tracking_config.yaml), named and titled after that directory. Multi-page PDF: cover with status lines → notes and AI Summary (when present) → analysis figures (per-phase when faceted) → statistical-comparisons table → structured experiment summary →

File	Contents
	QC figures (data quality plus per-tracker transitions/min and movement bars)

The experiment directory root also holds two **inputs** you may edit by hand: `tracking_config.yaml` (the design) and, when you have declared any, `removed_regions.yaml` (the regions you removed — §9.2.2). Neither is regenerated by a run.

qc/ — data quality

File	Contents
<code>*_data_quality.csv</code>	Per-tracker fraction of valid (non-missing) frames; trackers below <code>cutoff</code> are flagged

Project outputs (Projects only — at the Project root)

File	Contents
<code>analysis/<project>_Summary.csv / _Summary_Facet.csv</code>	The Combined Analysis: each replicate's <i>filtered</i> summaries stacked with an Experiment first column
<code>analysis/<project>_Excluded.csv</code>	All replicates' excluded flies, tagged by replicate
<code>analysis/<project>_Stats.txt</code>	Pooled per-fly Welch/Tukey tests beside the mixed-model p-values (treatment fixed, experiment random intercept), plus any cross-replicate warnings
<code>analysis/<project>_AI_Summary.txt</code>	(Optional) The AI narrative; deleted by every <code>build_combined_analysis()</code> and recreated by AI narrative...
<code>analysis/ai_narrative.md</code>	(Optional) The same narrative as Markdown, with front matter naming the level, model, and replicates. A fixed filename, so <code>**/ai_narrative.md</code> finds every narrative in a tree; written and deleted with its <code>.txt</code> sibling
<code>plot_specs.yaml</code>	Publication-figure Plot Specs + Plot Styles (written by the Plot Editor)
<code>figures/*.svg / *.pdf</code>	Vector publication figures rendered from the pooled data
<code><project>_report.pdf</code>	The Project Report: cover with per-replicate status → AI narrative (when present) → pooled publication figures → pooled + mixed statistics table → per-replicate summary table

8. Projects: replicates and combined analysis

A **Project** groups replicate experiment directories of one design under a parent directory marked by a **project.yaml** (layout and design rules in §3; [docs/adr/0005](#)). It replaces the retired batch-over-experiments mode: the Hub's Project panel runs every replicate, pools their results into a Combined Analysis, renders project-level publication figures, and builds a Project Report.

8.1 Creating a Project

- **New study — Project → Create/Load → Create project...**: pick/create the parent directory and edit the project **design** (experiment type, design factors and levels, facets, quality criteria, counting-region names). The design is seeded from the Experiment Type's defaults. An empty Project is valid — add replicates with **Experiments → Create experiment...**, which scaffolds each new **tracking_config.yaml** from the design.
- **Existing replicates / retired batch-over-experiments parent** — run **Initialize existing directory...** on the parent: it keeps its own name, its subdirectories become the replicates, the dialog infers the design from the first replicate that has a config, and **project.yaml** is written there; nothing inside the replicate directories changes.
- **Existing folders without configs** — the subdirectories are listed in the replicate table as **Config: missing. Experiment configs...** gives each one a **tracking_config.yaml** from the project design (**Create all missing** does the lot), then opens any of them in the Config Editor to assign that recording's regions. Existing configs are never overwritten. For a single folder, **Initialize existing directory...** does the same and also files a recording still sitting loose at its root into **data/**.

Opening a Project **hard-validates** every replicate's resolved config against the design and refuses to load on a mismatch, naming the offending replicate and key. Region → treatment assignments, counting-region aliases, fly counts, and rigs may differ; differing cutoffs or quality criteria are surfaced as warnings, not errors.

8.2 The Project panel workflow

With a Project selected, the Hub's **Project** panel (§5.2) holds the replicate table plus the project actions. For a full refresh, use **Create report** before **<project>_report.pdf** exists or **Update report** after it exists; both labels run every replicate, rebuild Combined Analysis, and write the Project report.

The remaining Project actions build on that refresh:

1. **Plot editor...** — curate the pooled publication figures (§5.5) from the Combined Analysis created by the report refresh. Save plot specs, then run **Update report** to rebuild the PDF with those specs.
2. **AI narrative...** — optional AI-written narrative for the Project Report (same rules as the per-experiment AI Summary: key-gated, clearly labeled, deleted by the next combined-analysis build). It rebuilds the Project report immediately so the PDF and saved text agree.

Double-click a replicate row to open it as the current experiment; the regular Analyze/Plots/Scripts/AI cards then apply to it, while the project actions above keep applying to the whole set. The two contexts are independent, so nothing has to be closed to get back to the Project. A report refresh rewrites every replicate's analysis and therefore unloads the current experiment rather than leave it holding results that no longer exist.

For anything more specific than the Hub's full report refresh, use a Project Script. Scripts expose the lower-level project steps — replicate analysis, Combined Analysis, publication figure rendering, report creation, AI

narrative, and `run_in_experiments` — without putting all of those partial actions back into the main Hub.

8.3 Project Scripts (two-level scripting)

Projects have their own saved scripts (`docs/adr/0006`) — same `{name, steps}` shape and the same visual editor as experiment scripts, but a separate **project-action** palette: `validate_design`, `run_in_experiments`, `project_report`, `render_publication_figures`, and `generate_ai_narrative` (soft-fail). They live under `scripts:` in `project.yaml`; the levels cannot mix. Each action mirrors a Project-card button, which is why `project_report` is the whole **Create report** sequence — analyze every replicate, pool, then render the PDF — rather than three separate steps. Scripts saved when `run_all_analyses` and `build_combined_analysis` were their own actions still run: both are absorbed into `project_report` at load.

The one bridge to experiment level is `run_in_experiments(script: NAME)`: it runs the named *experiment-level* script in every replicate — resolved first from the Project's central `experiment_scripts:` section (one recipe for all replicates, never copied into their configs), falling back to a script of that name in each replicate's own `tracking_config.yaml`. A legacy batch parent therefore still works: `run_in_experiments(script: batch)` runs the old per-folder `batch` scripts unchanged. Execution is replicate-by-replicate with per-replicate log prefixes and continue-on-error.

`run_in_experiments` also takes an optional `only:` parameter — a list of replicate directory names; blank means every replicate. The Script Editor renders `only:` as a checkable replicate list when editing a `project.yaml`. A name matching no replicate is logged and counted in the failure summary while the run continues; the Hub additionally **pre-checks** scripts before running, so unknown `only:` names or a script name that resolves nowhere abort with a message before anything runs.

The Analysis card's **Script** picker always includes two built-ins — zero authoring gets a complete run. Neither is ever written to `project.yaml`, so both track the shipped default.

- **Standard pipeline** — `validate_design` → `project_report` → `render_publication_figures` (SVG).
- **Report pipeline** — `project_report` → `render_publication_figures` (SVG), the latter dropped for a Project with no `plot_specs.yaml`. This is the **Create report** button plus curated figures, and the default designated script for Batch Runs (§8.5); it omits the `validate_design` gate, which would fail Projects mid-migration.

Both put the figure step *after* the report: `project_report` is what analyzes the replicates, and `render_publication_figures` reads their saved summaries, so the other order fails on a Project nobody has analyzed yet. **Edit scripts...** opens the Script Editor on `project.yaml` with the level switcher (§5.3).

8.4 Fixed pipeline from Python

```
from pytrackinganalysis.Experiment import batch_analyze

results = batch_analyze("./ParentFolder/") # {path: 'ok' | error message}
```

The legacy `batch_analyze` helper runs `run_analysis()` + `create_report()` on every immediate sub-directory that contains a `tracking_config.yaml` and a `data/` folder with at least one `.xlsx`; other

directories are skipped. Optional arguments: **cutoffs** (override every experiment's facet cutoffs), **qc_cutoff** (default 0.9), and **force_preprocessing**. It needs no **project.yaml** and builds nothing at the parent level — for the pooled Combined Analysis and Project Report, use the **Project** API (§6) or the Hub's Project panel; to run many *Projects* at once, use a Batch Run (§8.5).

Preparing many folders at once: the Hub's **Batch tools** dialog (copy one master YAML into every sub-directory, bulk-rename sub-directories, convert flat layouts into the **data/** structure, combine summary CSVs) was removed in favour of the Project workflow: **Experiment configs...** scaffolds a design-conformant config per replicate (§8.2), **Create experiment...** creates the **data/** structure for a new one, and Combined Analysis pools the summary CSVs (§6).

8.5 Batch Runs: many Projects at once

A **Batch** is structural: a directory with Projects anywhere beneath it — found by a recursive walk that stops at each Project (**docs/adr/0009**) — the Project ↔ experiment rule one level up. Nothing marks it, and it is a processing convenience only: a Batch never pools results across Projects (each keeps its own design and outputs), and its only artifact of its own is an optional **batch.yaml**.

Selecting such a folder in the Hub lights the leftmost **Batch** tile (§5.2). Its panel lists the Projects in a checkable table, a **Script** picker names the **designated Project Script**, and **Run batch** executes that script in every checked Project — sequentially in name order, continue-on-error, with per-Project log prefixes and a per-Project summary at the end (the **run_in_experiments** semantics one level up). The run unloads the loaded experiment first, because it rewrites every replicate's analysis. Sub-directories without a **project.yaml** are skipped with a log line — a Batch Run **never creates or upgrades a project.yaml**. There is no third script level: the thing a Batch Run runs *is* a Project Script.

The designation resolves per Project, in order:

1. **batch.yaml**'s **project_scripts:** section — Project Scripts held centrally at the Batch root so one recipe serves every Project (the **experiment_scripts:** idea one level up);
2. the Project's own **scripts:** section;
3. the built-ins (**Report pipeline, Standard pipeline**).

A name that resolves nowhere fails that Project; the run continues and the summary says so.

The default designation is the built-in **Report pipeline** (§8.3): **project_report** → **render_publication_figures**, the figure step dropped for a Project with no **plot_specs.yaml**. Zero authoring therefore makes a Batch Run mean "press **Create report** on every Project", without rendering uncurated default-spec figures unattended. **batch.yaml** is lazy — it is written only when the designation is changed away from that default (a **script:** key) or a **project_scripts:** section is authored by hand; leaving the default selected never creates the file.

The Batch AI narrative

The Batch panel also carries an opt-in **AI narrative of the batch** checkbox. With it ticked (the provider is chosen *before* the run starts), the Batch Run finishes by reading every Project's **analysis/ai_narrative.md** and asking the provider to synthesize them into **batch_ai_narrative.md** at the Batch root: results across the batch, compromised designs, and Projects that lost many flies — deliberately skipping the per-Project detail that lives one level down. It is a **synthesis, not a pooling**; a Batch never combines data across Projects.

Because the default `batch` script rebuilds each Combined Analysis, and that deletes the Project's narrative, most Projects have none left by the time the batch narrative is written — so a Project without one gets a narrative generated first (an extra provider call, logged as one). A Project that cannot produce one is named and skipped, and the front matter of `batch_ai_narrative.md` records which Projects the prose actually covers.

Results

Per-replicate results always land in each experiment's own `analysis/` and `qc/` folders; the pooled artifacts land at the Project root (§7) — the two levels never mix outputs. A Batch Run writes only into each Project the same way: the Batch folder itself gains at most a `batch.yaml` and, when the narrative was requested, a `batch_ai_narrative.md`; no batch-level `analysis` outputs exist.

9. Removed regions: excluding flies you observed

Some flies are lost in ways no automatic criterion can detect. A fly dies forty minutes into a seventy-minute recording, escapes while the plate is loaded, or the well was empty from the start. The tracker keeps producing rows regardless — a corpse is still a blob with a position — so the pipeline has no way to know. Only the person at the rig knows, which is why a **Removed Region** is declared by hand.

Two automatic mechanisms already act on the analysis population, and a removal is a third, deliberately different thing:

Mechanism	Who decides	Effect
Low-transition exclusion (§4.2)	The pipeline, from <code>min_transitions</code>	Removes the fly from every result
Low-movement flag (§4.2)	The pipeline, from <code>min_movement</code>	Reports the fly; never removes it
Removed region	You, from what you saw	Removes every fly in the region from every result

Removals are **not** a Valence feature. The low-transition exclusion is Valence policy, but a dead fly is a fact about the recording, so a removal applies to any experiment type — `TRACKER`, `TWOCHOICECOUNTER`, a Custom experiment with no type at all.

9.1 What a removal means

The unit is the tracking region, not the fly. You tick `T_14`, not `T_14_0`. A region is what you actually observe at the rig, it is already the addressable key in `tracking_regions:`, and it survives a DTrack re-export that rennumbers object IDs. Every tracker in the region goes: for the usual one-fly-per-well plate that is exactly one, and for a counter-class experiment (where a "tracker" is the region itself) it is the counter. A well holding two animals loses both — the well is what you declared dead.

Region matching is on the underscore boundary, so `T_1` removes `T_1` and `T_1_0` but never `T_10_0`, the well next door.

Removal is all-or-nothing. There is no "exclude only after minute 45". A dead animal still registers as occupancy wherever it died, so its Preference Index is dragged toward ± 1 and its Transitions count collapses — the numbers are unreliable for the *whole* recording, not just the tail. Censoring at a time of death would also give different flies different observation windows, which would silently compare sixty-minute flies against thirty-five-minute ones in the pooled tests and the mixed model.

Every removal carries a reason. Free text, in your words: **dead at ~20 min, escaped during transfer, well flooded, never loaded**. A removal entered without a reason is recorded as **Undefined** rather than as nothing, because the reason is what makes the audit trail readable six months later.

9.2 Declaring a removal

Three ways in, one source of truth. The Hub window and the removal sheet both end up writing the same file: **removed_regions.yaml** inside the experiment directory.

9.2.1 The Hub: Project → Removed regions...

The everyday path. Select the Project, open the **Project** tile, and press **Removed regions...** in the Analysis card. (From a Batch, **right-click** a row in the projects table and choose *Removed regions in ...* — double-click is already taken: it selects that Project.)

The window lists every tracking region of every replicate in the Project:

Column	Meaning
Remove	Tick to remove; already-declared regions open ticked
Experiment	The replicate directory name
Region	The region key as tracking_config.yaml spells it
Treatment	That region's experimental_factors , so you can see what you are dropping
Data	no data when the replicate has been analyzed and that region produced no tracker at all — an empty well. Blank for a replicate that has not been analyzed (nothing is known yet), and blank for a region that produced a fly, <i>including one that is currently excluded</i>
Reason	Free text, editable. Ticked and left blank ⇒ Undefined

Use **Filter** to narrow to one replicate (**Rep3**) or one region (**T_14**), and **Show removed only** to review what is already declared. Only regions the config declares are listed — the config's plate is the addressable set, and reading it costs no raw-data parsing, so a Project of eighty replicates opens instantly.

Save writes one **removed_regions.yaml** per replicate you changed, and logs each file in the Hub output. **Cancel** discards everything. Un-ticking a region deletes its declaration — that is the only way to un-remove a fly, and it is deliberately an explicit act.

If the experiment you have loaded in the Hub is one of the ones you changed, its Arena is re-filtered immediately: press a plot button straight afterwards and the removed fly is already gone. Saved results on disk are a separate matter — see §9.3.4.

9.2.2 The file: `removed_regions.yaml`

The declaration lives at the **experiment directory root**, beside `tracking_config.yaml`:

```
Rep3/
├── tracking_config.yaml    ← the design specification
├── removed_regions.yaml   ← what happened during the run
├── data/
└── analysis/
```

```
# Tracking regions the experimenter removed from the analysis (ADR-0010).
removed_regions:
  T_14: dead at ~20 min
  T_22: escaped during transfer
  T_31: Undefined
```

Rules:

- The keys must be region names exactly as `tracking_config.yaml` spells them. A key matching no region is **warned about and ignored** — never fatal (§9.3.5).
- A value may be omitted (`T_31:`); it reads back as `Undefined`.
- Deleting the file, or removing every entry from it, un-removes everything. The Hub deletes the file when you untick the last region, so a sidecar never outlives what it declared.
- The file travels with the recording. Move **Rep3** into another Project, copy it to another machine, and its removals come with it.

It is deliberately **not** part of `tracking_config.yaml`. That file is the design specification — what the experiment *is* — and is validated against the Project's shared **design:** section. How a particular run turned out is a different kind of fact, and mixing the two would mean every dead fly edits the design.

Editing this file by hand is fully supported: the window and the file are the same thing, and nothing caches it between runs.

9.2.3 The removal sheet: many projects at once

When you run ten Projects through a Batch, opening ten windows is not the workflow. Keep the notes where lab notes already live — a spreadsheet — and let the app distribute them.

Put `removed_regions.csv` (or `.xlsx`) at the **batch folder** root, or at a **Project** root:

project	experiment	region	reason
Starved2026	Rep3	T_14	dead at ~20 min
Starved2026	Rep3	T_22	escaped
Fed2026	Rep1	T_2	well never loaded

- **project** is needed only at a batch folder; at a Project root, omit it (a present-but-ignored column is fine too). It is the Project's path *relative to the batch folder*, so a Project sitting under grouping folders is written the way the Batch itself names it — **Sept2026/ProjA** — and a top-level one is just **ProjA**.
- Headers are matched case-insensitively and ignoring spaces and underscores, and common synonyms are accepted: **replicate** for **experiment**, **tracking region** / **well** / **tube** for **region**, **note** / **comment** for **reason**.
- A blank or missing **reason** becomes **Undefined**.
- A sheet missing the **experiment** or **region** column is rejected outright — an unparseable sheet is a mistake worth stopping for.

The sheet is a writer, not a second source of truth. Applying it stamps its rows into each experiment's **removed_regions.yaml**; nothing reads the sheet at analysis time. That is what keeps a Project reproducible: copy it to another drive without the spreadsheet and it still knows which flies you removed and why. (An overlay read at analysis time would silently return those flies to the results, with nothing in the Project recording that they were ever out.)

It is applied at exactly three moments, all of them deliberate:

1. **At the start of a Batch Run**, before the first Project script runs, so an unattended overnight run honours the notes you left.
2. When you press **Apply removal sheet...** in the Batch panel (the small button beside *Choose batch folder...*; it stays greyed out until the chosen folder actually holds a sheet).
3. When you open **Removed regions...** on a Project that has a sheet at its root — you are asked first.

Selecting a folder never applies anything on its own. Browsing to a colleague's batch folder must not rewrite eighty of their experiment directories.

Merge rules. Applying is additive and safe to repeat:

Situation	What happens	Reported as
Region not yet declared	Written with the sheet's reason	applied
Already declared, same reason	Nothing	already declared
Already declared, different reason	The standing reason is kept	conflict
Row names a project / experiment / region that does not exist	Nothing	unknown project / unknown experiment / unknown region
Row missing an experiment or region	Nothing	incomplete

The standing declaration wins because the sheet is re-applied on *every* Batch Run: letting the sheet win would keep resetting a wording you refined in the window. Conflicts are reported loudly — in the Hub they also raise a warning dialog — so a disagreement is never silent. Deleting a row from the sheet does **not** un-remove anything; untick it in the window instead.

Nothing in this path is ever fatal. A row that matches nothing is counted and logged; a sheet that cannot be read at all is reported and the Batch Run continues. One stale note must not kill ten Projects' worth of

overnight work.

9.3 What a removal does to the analysis

9.3.1 One choke point

Removed flies are merged with whatever the experiment type's own criterion excluded, and the combined set of tracker names is handed to **Arena** once, at load. `summarize()` and `summarize_facet()` drop those rows on the way out, and every consumer — plots, statistics, CSVs — reads through those two methods. There is no second place to forget, and no way for a figure and a table to disagree about who was in the analysis.

Filtered (the removed fly is absent):

- `*_Summary.csv` and `*_Summary_Facet.csv`
- every plot the Hub or the API draws, and the report's analysis figures
- `*_Stats.txt` — the pairwise tests are computed on the kept flies only
- the Project's Combined Analysis, which stacks the replicates' *filtered* summaries, and therefore the pooled statistics, the mixed model, and the publication figures rendered from them

Not filtered (the removed fly is still shown):

- everything under `qc/`, the QC figures in the report, and the QC Viewer. Quality control describes the *recording*, not the analysis population — if a well tracked badly you still want to see it
- the per-tracker QC bars in the report, which deliberately show every tracker
- the exclusion audit itself (§9.4)

9.3.2 Interaction with the automatic criteria

- The two are independent. `min_transitions: 0` turns the low-transition exclusion off; your removals still apply. Off means off for the *policy*, not for you.
- A fly caught by both appears **once**, with a reason naming both causes, the observation first: **Removed: dead at ~20 min; Low transitions**. Counts never double.
- The **low-movement flag** is computed on the analysis population *after* exclusions, so removed flies are not flagged and do not count toward the ">50 % of flies flagged ⇒ the experiment is potentially an issue" rule. Removing three dead wells therefore makes that rule stricter, not laxer.
- A **Custom** experiment has no automatic criterion at all and produced no `*_Excluded.csv` before; declare a removal and it gets one, with a narrower schema (no **Transitions** column, because nothing computes transitions there). An experiment with no removals declared is untouched.

9.3.3 Fly counts

Flies in the replicate table and the reports is the count *after* exclusions — the analysis population. The arithmetic is always `analyzed = tracked - excluded`, and **excluded** is one number covering both causes (§9.4).

9.3.4 Removals declared after an analysis

Declaring a removal does not rewrite results that are already on disk. The pipeline notices:

- A replicate is **stale** when the regions in its `removed_regions.yaml` disagree with the removal rows in its saved `*_Excluded.csv`. This is a content comparison, not a timestamp one, so it survives copying a Project between drives.
- The Hub's replicate table shows **re-run needed** in the **Excluded** column, and the Project report marks the row red.
- A Project Script step told to **skip analyzed replicates** re-runs a stale one anyway, and says why in the log. Without that, an unattended run with `skip_analyzed: true` would happily pool data you had already thrown out.
- The Hub's **Create/Update report** and the default `batch` script re-analyze every replicate regardless, so in the ordinary batch workflow the removals are picked up on the next run with nothing to remember.

The loaded experiment in the Hub is the exception that needs no re-run: saving the removals window re-filters its Arena in memory immediately.

9.3.5 Declarations that match nothing

A region in `removed_regions.yaml` that is not a region of that experiment — a typo, or a plate that changed under an old note — removes nothing. It is reported everywhere it matters and aborts nothing:

- a warning line when the experiment loads, and in the `*_Stats.txt` preamble
- a footnote in the experiment report: *"T_40 was declared removed but matches no tracking region in this experiment — nothing was excluded for it."*
- for sheets, a per-row **unknown region** (or **unknown experiment / unknown project**) in the application log, with the totals summarised on one line

The report footnote matters more than the log line: after an overnight batch, the report is the artifact anyone actually reads.

9.4 How removals are reported

Nothing is removed quietly. A removal that no reader can see is indistinguishable from data going missing, so the audit runs from the console line at load all the way to the Project report.

At load, and in the Hub output:

```
Excluded 4 fly(ies): 3 removed by the experimenter, 1 with fewer than 5
transitions during the Experiment phase.
Warning: removed region T_40 matches no tracker in this experiment
(declared in removed_regions.yaml).
```

analysis/<name>_Excluded.csv — one row per fly, written even when empty so its absence never needs interpreting:

Name	TrackingRegion	Treatment	Transitions	Reason
T_14_0	T_14	Starved	2	Removed: dead at ~20 min; Low transitions

Name	TrackingRegion	Treatment	Transitions	Reason
T_22_0	T_22	Control	31	Removed: escaped during transfer
T_5_0	T_5	Starved	1	Low transitions

The **Reason** grammar is worth knowing:

- **Removed: <your text>** — your declaration. The **Removed:** prefix is what keeps a reason of **Undefined** from being mistaken for a machine verdict.
- **Low transitions** — the automatic Valence criterion; the **Transitions** column holds the count that failed, or **no data** when the fly had none in the primary phase.
- **Removed: ...; Low transitions** — both applied, your observation first, because an observation outranks an inferred rule ("I saw it dead" explains "few transitions", not the other way round).
- A Custom experiment's file has no **Transitions** column at all.
- Files written before this feature existed have no **Reason** column; they read as **(not recorded)** where one is needed, rather than being rewritten.

analysis/<name>_Stats.txt, in the preamble that orients the reader before any p-value:

```
Excluded      : 4 fly(ies) – 3 removed by the experimenter; 1 with fewer
than 5 transitions during the Experiment phase (see _Excluded.csv)
               warning: removed region T_40 matches no tracker in this
experiment.
```

The experiment report (<name>_report.pdf):

- the cover's Data overview reads **Excluded flies: 4 (3 removed, 1 low transitions)**
- an **Excluded flies** section opens the Analysis part of the report, before any result computed from the flies that remained: a sentence giving the counts by cause, then a table of **Fly | Region | Treatment | Transitions (phase) | Reason**, then a footnote per unmatched declaration
- with the low-transition exclusion off and removals present, the section says so explicitly rather than claiming nothing was excluded
- QC figures further down still show every tracker, removed ones included

The Project report (<project>_report.pdf):

- the **Per-replicate summary** table's **Excluded** cell reads **7 (4 removed)**, and **re-run needed** when a replicate's declarations have not reached its saved analysis; such rows are marked red
- an **Excluded flies** table lists *every* excluded fly across the Project — **Experiment | Fly | Region | Treatment | Reason** — so one page answers "what did we lose, where, and why"
- when nothing was excluded anywhere, one sentence says so

The Hub: the replicate table's **Exc luded** column carries the same **7 (4 removed) / re-run needed** text as the report.

AI summaries: ***_Exc luded.csv** is part of what an AI Summary is given, so the reasons reach the experiment narrative, and through each Project's narrative, the Batch AI Narrative — which is why it can say *why* a Project lost flies, not just how many.

9.5 Worked examples

One dead fly in one replicate

1. Project tile → **Removed regions...**
2. Filter to **Rep3**, tick **T_14**, type **dead at ~20 min**, **Save**.
3. Press **Create/Update report** (or run the Project's **batch** script). The replicate is re-analyzed, the fly is gone from every figure and statistic, and both reports name it.

Ten Projects from lab notes

1. In Excel, list every loss as **project, experiment, region, reason**; save as **removed_regions.csv** at the batch folder.
2. Batch tile → **Choose batch folder...** The log reports the sheet and its row count; nothing is written yet.
3. Either press **Apply removal sheet...** to write the declarations now and inspect them, or just press **Run batch** — the sheet is applied first automatically.
4. Read the **[removals]** lines: **applied, already declared, conflict**, and any **unknown ...** rows that matched nothing.

Undoing a removal

Open **Removed regions...**, untick the region, **Save**, then re-run the analysis. Deleting the row from the removal sheet does nothing — the sheet only ever adds.

Auditing what is currently removed

Every declaration is a small YAML file at a known path, so the whole batch is one command away:

```
# Every declaration in a batch folder, file by file
grep -r -A20 "^removed_regions:" --include=removed_regions.yaml .

# Or read the audit the last analysis produced
column -s, -t < Starved2026/Rep3/analysis/Rec_Excluded.csv
```

9.6 Rules of thumb

- **Removing flies changes published numbers.** That is the point, and it is why every removal is written down with a reason in three places. Re-run the analysis after declaring one; the reports will tell you if you forgot.
- **Do not use removals for problems the pipeline can already see.** A badly tracked well is a quality-control matter (**qc/**, the low-movement flag); a fly that simply sat still is what **min_transitions** is for. Removals are for facts only you have: death, escape, an empty well, a mis-loaded plate.
- **Write the reason as you would in a lab notebook.** **dead at ~20 min, tube leaked** is worth far more in a year than **dead**, and it costs nothing.
- **Region names must match the config exactly** — copy them from the window or the config, not from memory.

- **A removal is not censoring.** If you genuinely need "the first forty minutes of this fly", that is a facet-window question, not a removal.

10. Quick reference

Starting the UI

```
# Analysis Hub (the default entry point):
pytrack                                # shorthand for pytrack-hub
pytrack-hub /absolute/path/to/project

# Via uv (works from any directory, no activation needed):
uv run pytrack-hub /absolute/path/to/project

# Standalone Config Editor (with visual Script Editor):
pytrack-config
pytrack-config /path/to/ExperimentDirectory

# Standalone QC Viewer:
pytrack-qc /path/to/ExperimentDirectory

# Plot Editor (publication figures; Project directories only):
pytrack-plots /path/to/MyProject

# One-time: install launcher entries + taskbar icon (Linux):
pytrack-install-desktop
```

Minimal tracking_config.yaml

```
global:
  tracking_type: TWOCHOICETRACKER
  tracking_rig: colosseum
  experimental_design_factors:
    treatment: [A, B]

tracking_regions:
  T_0:
    experimental_factors: A
    x_location_multiplier: 1
    y_location_multiplier: 1
  T_1:
    experimental_factors: B
    x_location_multiplier: 1
    y_location_multiplier: 1
```

One-line pipeline (Python)

```
from pytrackinganalysis.Experiment import Experiment
exp = Experiment("/path/to/experiment/")
exp.run_analysis()
exp.create_report()
```

Project pipeline (Python)

```
from pytrackinganalysis.project import Project
prj = Project("/path/to/MyProject/")
prj.run_all()
prj.build_combined_analysis()
prj.create_report()
```

AI features

Put **ANTHROPIC_API_KEY** and/or **OPENAI_API_KEY** in a **.env** file (next to where you launch from, or **~/config/pytrackinganalysis/.env**). The Hub's **AI** card (per-experiment summary) and the Analysis card's **AI narrative...** then light up; without a key they stay disabled and everything else works unchanged.

Rig calibration values

tracking_rig	mm / pixel
small_arena	0.056
arena_max	0.145
colosseum	0.108
obscura	0.131
movie	user-supplied

Default analysis parameters

Parameter	Default	YAML key
Speed smoothing window	1 second	speed_window_seconds
Micro-movement range	0.2 – 2 mm/s	micromove_speed_mm_sec
Walking threshold	2 mm/s	walking_speed_mm_sec
Sleep threshold	5 min continuous rest	sleep_threshold_min
Interaction distance	8 mm	interaction_distances

Environment commands

```
uv sync                # install / update all dependencies
uv add <package>       # add a new dependency
uv run python <script>  # run without activating the environment
source .venv/bin/activate # activate on macOS / Linux
.venv\Scripts\Activate.ps1 # activate on Windows PowerShell
```